Checkpoint — MLflow Integration & Drift-Triggered Retraining

Real-Time User Behavior Anomaly Detection Platform
Amen Bank — DCSI

Yassine Ben Jemia

July 24, 2026

1 Session Objectives

1. Integrate MLflow experiment tracking into the training pipeline with a promotion gate.
2. Build a drift-triggered retraining subsystem: Prometheus → AlertManager → webhook relay → Airflow.
3. Add a feedback divergence monitor as a separate Airflow DAG.

2 MLflow Integration

2.1 Design Decisions

- **Backend:** Flat-file storage (`mlruns/` volume mount). The Model Registry requires a database backend, which was deemed unnecessary at the project's scale (weekly retrains, <10 total runs before deadline).
- **Experiment structure:** Two independent runs per training execution (one for AE, one for LSTM) under a single experiment `amen-anomaly-detection`. Shared artifacts (scaler, feature columns) are duplicated across runs for self-containment.
- **Promotion gate:** Each model is independently compared against `models/production_metrics.json`. Promotion requires $new\_auc \geq production\_auc$. The $\geq$ (not $>$) ensures fresh models trained on fresh data are preferred even at equal AUC.
- **MLflow's role:** Passive logging only. MLflow does not decide, compare, or promote — all gate logic lives in `train_all.py`. MLflow's value is post-mortem debugging: when a gate rejects a model at 4 AM Sunday, the engineer can inspect parameters, metrics, and artifacts on Monday.

2.2 Logged Data Per Run

Category	Fields	Example
Parameters	hidden_dims, epochs, batch_size, lr, model_type n_features, n_train_samples	`lr=1e-3` `n_features=30`
Metrics	test_auc, production_auc, promoted	`test_auc=0.9663`
Tags	status	`promoted` or `rejected`
Artifacts	scaler.pkl, feature_cols.json	duplicated per run

2.3 Docker Service

MLflow server runs as a standalone container (`ghcr.io/mlflow/mlflow:v2.15.1`) on port 5001 (host) → 5000 (container). Training container connects via `MLFLOW_TRACKING_URI=http://mlflow:5000`.

2.4 Validation Results

First training run completed successfully. Both models promoted (compared against 0.0 baseline due to missing bootstrap file — noted as a one-time issue):

Model	Test AUC	Prod AUC	Status
Autoencoder	0.9663	0.0000	promoted
LSTM	0.9340	0.0000	promoted

`production_metrics.json` now bootstrapped with these values for subsequent runs.

3 Drift-Triggered Retraining Subsystem

3.1 Architecture

The retraining trigger chain follows the industry-standard observability pattern:

`Prometheus` *alertrule* `AlertManager` *webhook* `webhook-relay` *RESTAPI* `Airflow` *DAG* `train_all.py` *logging* `MLflow`

3.2 Prometheus Alert Rule

- **Metric:** `fused_score_distribution` histogram (already instrumented in Scoring Service).
- **Condition:** Fraction of events scoring above T1 (0.95) exceeds 15%.
- **Duration:** `for: 1h` — condition must hold continuously for one hour to filter out genuine fraud bursts.
- **File:** `config/alert_rules.yml`, loaded via `rule_files` in `prometheus.yml`.

3.3 AlertManager

Receives firing alerts from Prometheus, routes to `webhook-relay` via HTTP POST. Key parameters:

- `group_wait: 10s` — initial wait before sending first notification.
- `repeat_interval: 24h` — if drift persists after retraining, no re-trigger for 24 hours.
- `send_resolved: false` — no action needed when drift resolves.

3.4 Webhook Relay

A lightweight FastAPI service (`src/webhook/relay.py`, 20 lines) that translates AlertManager's JSON payload into an Airflow REST API call. Uses a dedicated `Dockerfile.webhook` based on `python:3.11-slim` (∼150 MB vs 2+ GB for the PyTorch-based `Dockerfile.api`).

Airflow REST API enabled via `AIRFLOW__API__AUTH_BACKENDS=airflow.api.auth.backend.basic_auth`.

3.5 E2E Validation

Webhook relay tested from inside Docker network:

```
POST http://localhost:8000/trigger -> 200
Airflow response: dag_run_id=manual__2026-07-24T12:22:43
                  state=queued, external_trigger=true
                  conf={"trigger": "drift_alert"}
```

4 Feedback Divergence Monitor

4.1 Design

A separate Airflow DAG (`dags/feedback_monitor.py`) runs daily and queries PostgreSQL for supervisor rejection rates on ALERT + BLOCK tier events over the past 7 days.

- **Trigger condition:** Rejection rate $> 50\%$ with minimum 10 reviewed decisions.
- **Scope:** ALERT + BLOCK tiers only. REVIEW excluded because higher false-positive rates at that tier are expected by design.
- **Minimum sample size:** 10 reviews prevents triggering on insufficient data ($\sim$2 days of ALERT/BLOCK volume at current rates of $\sim$5/day).
- **Mechanism:** Uses `airflow.api.common.trigger_dag` to trigger `weekly_retrain` programmatically with `conf={"trigger": "feedback_divergence"}`.

4.2 Retrain Trigger Summary

Trigger	Detection	Mechanism	Tempo
Scheduled	Time-based	Airflow cron (Sunday 4 AM)	Weekly
Drift	Score distribution shift	Prometheus → AlertManager → webhook	Real-time
Feedback	Supervisor disagreement	Airflow DAG (daily SQL check)	Daily

5 Docker Compose Changes

Service count: 17 → 20 (added `mlflow`, `alertmanager`, `webhook-relay`).

Service	Image	Port
mlflow	ghcr.io/mlflow/mlflow:v2.15.1	5001
alertmanager	prom/alertmanager:v0.27.0	9093
webhook-relay	python:3.11-slim (custom)	internal only

6 Files Created / Modified

File	Action
`src/training/train_all.py`	Modified — MLflow tracking + promotion gate
`src/webhook/__init__.py`	Created — empty
`src/webhook/relay.py`	Created — AlertManager → Airflow bridge

`config/alert_rules.yml`	Created — Prometheus drift detection rule
`config/alertmanager.yml`	Created — AlertManager routing config
`config/prometheus.yml`	Modified — added rule_files + alerting sections
`models/production_metrics.json`	Created — promotion gate baseline
`dags/feedback_monitor.py`	Created — daily feedback divergence check
`Dockerfile.webhook`	Created — lightweight relay image
`docker-compose.yml`	Modified — 3 new services, Airflow API env var
`requirements.txt`	Modified — added mlflow

7 Next Session

1. Dead letter queue (DLQ) for failed events in the streaming pipeline.
2. Structured JSON logging across all services.
3. Rapport de stage + soutenance preparation.